Лабораторна робота № 6

Тема. Застосування оброблення винятків і контекстних менеджерів.

Мета: навчитися застосовувати механізми оброблення винятків та контекстні менеджери.

Короткі теоретичні відомості

Виняток – це помилка, яка виникає під час виконання програми та порушує її нормальне виконання. Без обробки винятків програма просто завершує роботу та виводить traceback (трасування помилки). Приклад основного механізму обробки винятків через блоки *try* та *except*:

```
try:
    x = 10 / 0 # помилка
except:
    print("Ділення на нуль!")
```

Приклад з уточненням помилки:

```
try:
    num = int(input("Введіть число:"))
    result = 100 / num
    print(f"Результат: {result}")
except ValueError:
    print("Помилка: Ви ввели не число!")
except ZeroDivisionError:
    print("Помилка: Ділення на нуль!")
```

ValueError виникає, коли функція або операція отримує аргумент правильного типу, але з неприпустимим значенням.

ZeroDivisionError виникає під час спроби виконати ділення на нуль.

Блок *else* виконується тільки якщо в *try* не виникло помилки. Блок *finally* виконується завжди, незалежно від того була помилка чи ні. Приклад використання блоків *else* та *finally*:

```
try:
    num = int(input("Введіть число:"))
    result = 100 / num
    print(f"Результат: {result}")
except ValueError:
    print("Помилка: Ви ввели не число!")
except ZeroDivisionError:
```

```
    print("Помилка: Ділення на нуль!")
else:
    print("Операція успішно виконана. Результат:", result)
finally:
    print("Операція завершена.")
```

Приклад виклику помилки через оператор *raise*:

```
def set_age(age):
    if age < 0:
        raise ValueError("Вік не може бути від'ємним!")
    print(f"Ваш вік: {age}")

try:
    set_age(-5)
except ValueError as e:
    print(f"Перехоплено помилку: {e}")
```

Приклад створення власного класу винятку, який успадковується від стандартного класу *Exception*:

```
class InsufficientFundsError(Exception):
    pass

class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def withdraw(self, amount):
        if amount > self.__balance:
            raise InsufficientFundsError("Недостатньо коштів.
Доступно: {self.__balance}")
        self.__balance -= amount
        print(f"Знято {amount}. Залишок: {self.__balance}")

try:
    acc = BankAccount(100)
    acc.withdraw(150)
except InsufficientFundsError as e:
    print(f"Помилка операції: {e}")
```

Приклади поширених вбудованих винятків:

Тип винятку	Виникнення
ZeroDivisionError	Ділення на нуль.
ValueError	Правильний тип, але некоректне значення.
TypeError	Операція застосована до несумісних типів.
KeyError	Ключа немає у словнику.
AttributeError	Об’єкт не має такого атрибута/методу.
IndexError	Індекс поза межами списку.
StopIteration	Використовується для повідомлення про відсутність наступних елементів в ітераторі.

Контекстний менеджер – це спеціальний механізм, який дозволяє чітко визначити, що має статися на початку виконання блоку коду та що обов’язково має статися в кінці (навіть якщо виникла помилка). Він реалізується за допомогою оператора *with*. Головна мета використання контекстних менеджерів – це гарантоване звільнення ресурсів. Без контекстних менеджерів потрібно було б писати багаторазові конструкції *try/catch*, що робить код громіздким. Для того, щоб об’єкт став контекстним менеджером, він повинен реалізувати два спеціальні методи: *__enter__(self)* викликається на початку блоку *with* (перед виконанням коду) виконує налаштування та повертає об’єкт, який записується у змінну після *as*; *__exit__(self, exc_type, exc_val, exc_tb)* викликається у кінці блоку *with* (навіть якщо сталася помилка), використовується для звільнення ресурсів (наприклад закриття файлів або завершення роботи з підключеннями), якщо метод повертає значення *True*, виняток вважається обробленим і не передається далі, тому таке використання потребує обережності. Приклад створення власного контекстного менеджера:

```
# модуль, який містить набір функцій для роботи з часом
import time

class Timer:
    def __enter__(self):
```

```
        self.start = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.end = time.time()
        duration = self.end - self.start
        print(f"Час виконання: {duration} секунд")
        return False

with Timer() as t:
    sum(range(1000000)) # імітація тривалої операції
```

Приклад створення власного контекстного менеджера через декоратор *@contextmanager* з модуля *contextlib*:

```
import time
from contextlib import contextmanager # імпортує декоратор з модуля

@contextmanager
def timer():
    start = time.time()
    try:
        yield # виконується код всередині блоку with
    finally:
        end = time.time()
        print(f"Час виконання: {end - start:.4f} секунд")

with timer():
    # _ використовується для розділення розрядів
    sum(range(1_000_000))
```

:.4f – специфікатор форматування, де *:* позначає початок опису формату, *.4* задає точність відображення числа (з чотирма знаками після десяткової крапки), а *f* задає виведення числа у форматі десяткового дробу.

Decimal – це клас із модуля *decimal*, який використовується для роботи з десятковими числами з високою точністю. Його використовують, коли звичайний тип *float* може давати помилки через особливості зберігання дробових чисел. Приклад:

```
print(0.1 + 0.2) # Вивід: 0.30000000000000004
```

Приклад використання *Decimal*:

```
from decimal import Decimal

print(Decimal("0.1") + Decimal("0.2")) # Вивід: 0.3
```

Основні переваги *Decimal*: точні обчислення з десятковими дробами; зручний для фінансів; можна контролювати кількість знаків після коми та правила округлення.

Порядок виконання роботи

1. Уважно вивчити короткі теоретичні відомості та літературу.
2. Реалізувати контекстний менеджер та власні винятки (якщо є) відповідно до власного варіанта.
3. Написати звіт з виконання лабораторної роботи.

Варіанти завдань

№ варіанта	Контекстний менеджер
1	Тимчасова конфігурація (*ConfigContext*)
2	Ігровий баф (*BuffContext*)
3	Перемикач координат (*PolarContext*)
4	Підвищення прав (*AdminMode*)
5	Логування методів (*DebugLog*)
6	Тимчасова заміна *__str__ ()* (*StrOverride*)
7	Додавання префікса до *print()* (*PrintPrefix*)

Контрольні питання

1. Яке основне призначення конструкції *with*?
2. Яке призначення методу *__enter__*?
3. Яке призначення методу *__exit__*?
4. Яке ключове слово використовується для того, щоб штучно згенерувати (викинути) виняток у коді?
5. Який вбудований виняток виникає, якщо спробувати поділити число на нуль?